Krishna Dalvi
D20C/10

Blockchain : Experiment - 1

Aim:

Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash.

Tasks Performed:

1. **Hash Generation using SHA-256:**
   Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
   Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
   Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
   Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program & Output:

Program #1: Python program that uses the hashlib library to create the hash of a given string:

XXXXXXX #error statement; 3 Error

```
import hashlib                                                  #error statement

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()                       #error statement
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)                                      #error statement
```

```
Enter a string: shakti ronit harsh
Hash: 800835bf0ef9454efaec66343d64c4296950b0a8834dbbda948bbae37be0bff0
```

Program #2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce                                    # Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)                                        # Error
```

```
Enter a string: shakti ronit harsh
Enter the nonce: 5
Hash Code: bf0b1b448f5ff10707edbd4179ce9232a66afebd34c1302cf7c9ae62c21344a6
```

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string 3 Error

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)                       #Error
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):                         #Error
            print("Hash:",hash_code )
            return nonce

        nonce += 1

# Get user input
input_string = "A"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)                              #Error
```

```
Hash: 06663975f75f189f1d70bd14d8f22df264cd6a5c7575d687518 3d0ec76432fbb
Input String: A
Leading Zeros: 1
Expected Nonce: 9
```

Program 4: Generating Merkle Tree for given set of Transactions

```
import hashlib

def build_merkle_tree(transactions):                                    # Error
    if len(transactions) == 0:
        return None

    if len(transactions) == 1:
        return transactions[0]

    # Recursive construction of the Merkle Tree
    while len(transactions) > 1:
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []
        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i+1]
            hash_combined = hashlib.sha256(combined.encode('utf-8')).hexdigest()
            new_transactions.append(hash_combined)                      # Error

        transactions = new_transactions

    return transactions[0]

# Example usage
transactions = [
    "Alice → Bob : $200",
    "Bob → Dave : $500",
    "Dave → Eve : $100",
    "Eve → Alice : $300",
    "Roo → Bob : $50"
]
merkle_root = build_merkle_tree(transactions)
print("Merkle Root:", merkle_root)                                      # Error
```

```
Merkle Root: e338807e9d0a48e9940b7d62e8b4b339dc55858549634f783114d2e7db031a33
```

All the intermediate hash during the construction of the Merkle Tree Root Hash:

```
def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None
    if len(transactions) == 1:
        return transactions[0]
    level = 0
    while len(transactions) > 1:
        print(f"\n--- Level {level} ---")
        print("Nodes:", transactions)
        # If odd number of nodes, duplicate the last node
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])
        new_transactions = []
        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i + 1]
            hash_combined = hashlib.sha256(
                combined.encode('utf-8')
            ).hexdigest()
            print(f"Hash of:")
            print(f"  {transactions[i]}")
            print(f"  {transactions[i + 1]}")
            print(f"  -> {hash_combined}\n")
            new_transactions.append(hash_combined)
        transactions = new_transactions
        level += 1
    print(f"--- Merkle Root ---")
    print(transactions[0])
    return transactions[0]

transactions = [
    "Alice → Bob : $200",
    "Bob → Dave : $500",
    "Dave → Eve : $100",
    "Eve → Alice : $300",
    "Roo → Bob : $50"
]

merkle_root = build_merkle_tree(transactions)

print("\nFinal Merkle Root:", merkle_root)
```

```
Hash of:
  Alice → Bob : $200
  Bob → Dave : $500
  -> dbf71b02027d709e21ca4d0a586724d0664412ef15e458f4f97f18bf0ead578b

Hash of:
  Dave → Eve : $100
  Eve → Alice : $300
  -> 247a5c5e28ee9ec7a4545614ba5704804b0b2d500562657513985b5e9104097c

Hash of:
  Roo → Bob : $50
  Roo → Bob : $50
  -> 1c38b690dffd0c9cd3d1f996eb8cbc2b06564dc747561eed43249a52271d9430
```

```
--- Level 1 ---
Nodes: ['dbf71b02027d709e21ca4d0a586724d0664412ef15e458f4f97f18bf0ead578b', '247a5c5e
Hash of:
  dbf71b02027d709e21ca4d0a586724d0664412ef15e458f4f97f18bf0ead578b
  247a5c5e28ee9ec7a4545614ba5704804b0b2d500562657513985b5e9104097c
  -> 4e8de27e4c5f12c0d27eda76920a517ecd902bc9a45ed61d56dd513a51dcbec6

Hash of:
  1c38b690dffd0c9cd3d1f996eb8cbc2b06564dc747561eed43249a52271d9430
  1c38b690dffd0c9cd3d1f996eb8cbc2b06564dc747561eed43249a52271d9430
  -> 8b32cf64ef5ab79e33bc54d5a83c3d330ace95eeb404401a677ebe1003cc1f17


--- Level 2 ---
Nodes: ['4e8de27e4c5f12c0d27eda76920a517ecd902bc9a45ed61d56dd513a51dcbec6', '8b32cf64
Hash of:
  4e8de27e4c5f12c0d27eda76920a517ecd902bc9a45ed61d56dd513a51dcbec6
  8b32cf64ef5ab79e33bc54d5a83c3d330ace95eeb404401a677ebe1003cc1f17
  -> e338807e9d0a48e9940b7d62e8b4b339dc55858549634f783114d2e7db031a33

--- Merkle Root ---
e338807e9d0a48e9940b7d62e8b4b339dc55858549634f783114d2e7db031a33

Final Merkle Root: e338807e9d0a48e9940b7d62e8b4b339dc55858549634f783114d2e7db031a33
```

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

1. SHA-256 Hashing provides a secure and unique fingerprint for any input, ensuring data integrity.
2. Nonce and Target Hashing allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
3. Proof-of-Work Puzzle Solving shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
4. Merkle Tree Construction efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.